

Vyomi Distribution Strategy

Goal

Make Vyomi easy to install and operate on macOS, Linux, and Windows using one of three familiar patterns:

- package managers,
- `docker compose`,
- or OS-native installers.

The platform should feel like a product, not a hand-wired stack of host processes.

Distribution Tiers

Vyomi ships as a **seven-tier ladder** (renamed from the earlier CloudMax/CloudLite+/`...` scheme in v2.2.0) along two axes: three runtime **substrates** (WASM → Docker → Multipass-LXD) × whether real **compute** is bundled. The names double as subscription tiers. They form a funnel: **Nano** (a URL, zero install) pulls users in; **Free/Lite/Pro** (`docker compose up`) convert developers; **Max** serves teams needing real VMs; **Enterprise** adds packs + on-prem. Price tracks substrate weight (WASM → Docker → Multipass), with +compute as the premium within each.

Tier	Price (₹/mo)	Substrate	Conformance	Compute
Free	0	Docker	full API/SDK/Utility	capped (1 instance, no SSH)
Nano	199	WASM (browser)	API/SDK/Utility only	— none
Micro	299	WASM (browser)	full	Pyodide · CheerPJ · TinyGo (in-tab)
Lite	399	Docker	API/SDK/Utility only	— none
Pro	499	Docker	full	EC2-on-Docker + SSH
Max	599	Multipass / LXD	full	real VMs
Enterprise	Talk to sales	Max + custom	full + packs	real VMs + add-on packs

Substrate maps to **how it installs** (not a forked codebase — see ADR-001): Free/Lite/Pro share the **docker compose up** front door (`docker-compose.cloudlite.yml`); Max uses the **Multipass launcher** (deb/rpm/scoop/msi); Nano/Micro are **in-browser** (a URL). The conformance pack is **constant** across all tiers — only its runtime form (container vs WASM) and whether compute is bundled differ, conditioned by a flag, never a separate flow.

System requirements

Tier	CPU	RAM	Disk	Virtualization	OS / install
Free	2 cores	4 GB (2 min)	~8 GB	yes (Docker Desktop/WSL2 → VT-x)	mac/Linux/Win · <code>docker compose up</code>
Nano	2 cores	4 GB	~200 MB cache	none	any modern browser · a URL
Micro	4 cores	8 GB	~1 GB cache	none	Chromium/Firefox (SharedArrayBuffer) · a URL
Lite	2 cores	4 GB	~5 GB	yes (Docker/WSL2 → VT-x)	mac/Linux/Win · <code>docker compose up</code>
Pro	4 cores	8 GB	~12 GB	yes (Docker/WSL2 → VT-x)	mac/Linux/Win · <code>compose / deb-rpm-scoop-msi</code>
Max	4 cores	16 GB (8 degraded)	~40 GB	yes (Hyper-V/VirtualBox → VT-x)	mac/Linux/Win · <code>brew-deb-rpm-scoop-msi</code>
Enterprise	custom	custom	custom	yes (deployment-dependent)	+ air-gapped / on-prem

Notes / confidence:

- **Max — 16 GB recommended, 8 GB is the degraded edge.** Validated 2026-06-22: an 8 GB Windows 10 Home laptop **froze** during the image pull because the auto-sizer handed the VM 4 GB, starving the host. Confidence: **high** (real failure data).
- **Nano/Micro are the only truly virtualization-free tiers** — they run entirely in the browser (WASM). The Docker tiers (Free/Lite/Pro) still need Docker Desktop, which runs on WSL2 (a VM) → VT-x on Windows. Confidence: **medium** (Docker measured; WASM tiers estimated, unbuilt).
- **Tier capability is enforced by the license** (`core/tier_policy.py`), not the installer. Free already locks `nosql / eventing` and caps `vm:1` ; that cap IS the "Free = partial Pro" line. Pro unlocks full compute + SSH.

ADR-001: Single codebase, tier-as-build-profile

Status: Accepted (2026-06-23)

Context. Five tiers (above) invite a tempting mistake: forking into separate codebases per tier. But Vyomi's entire value proposition is native-SDK conformance — *the same SDK code passes the same tests on every tier*. Separate cores would drift, a green light on one tier would say nothing about another, and every bug fix / new service would land five times.

Decision. One codebase. Tiers are produced by build profiles plus a runtime compute flag — never by forking.

1 codebase → 3 build targets (Max / Lite / Nano substrates) → 5 tiers (compute flag on/off).

Mechanism.

- **Across substrates (Max / Lite / Nano) → build profile.** These are genuinely different runtime targets (a Linux VM, a host-Docker stack, a WASM browser bundle); you cannot ship one artifact that is all three. Each substrate is a build target that packages the same core and bundles only that tier's seam implementations.
- **The + (compute on/off) → runtime flag.** Lite and Lite+ are the *same* artifact with the compute backend enabled/disabled by config; likewise Nano and Nano+. So three builds yield five tiers.
- **One shared conformance pack** — the simulator/handlers (cloud-API logic) are a single implementation, identical across all tiers.
- **Two seams carry all variation:** `ComputeBackend` (LXD / Docker / browser-runtime) and `BackendProvider` (real container / WASM-in-memory). Tier-specific code lives **only** in these adapters; the core never imports a tier.

Invariants (CI-enforced).

1. **WASM-clean core.** The shared core must compile/run in WASM (Pyodide). Native/heavy dependencies (real Postgres, LXD, etc.) live behind the seams, never in the core. *This is the rule that stops CloudNano becoming an accidental fork* — if native deps leak into the core, Nano is forced into a parallel implementation.
2. **Conformance parity.** The same native-SDK conformance suite runs on every tier. A backend with no implementation in a given substrate is marked *unsupported there* — never reported as a false green.
3. **No tier imports in the core.** A CI grep-gate asserts that core modules never reference tier-specific adapters.

Consequences.

- (+) A fix lands once; conformance is comparable across tiers; the funnel (Nano → Lite → Max) behaves consistently; five tiers from three builds.
- (–) The seams demand discipline, and the WASM-clean invariant constrains the core's library choices (push native concerns into adapters).
- (–) Three build targets + per-tier CI to maintain.

Rejected alternative — separate codebase per tier. 5× maintenance and drift; breaks conformance integrity (the core value prop); guarantees funnel inconsistency. Not viable.

ADR-002: Packs as composable modules

Status: Accepted — roadmap (Core Pack ships first) (2026-06-23)

Context. Beyond the runtime tiers, Vyomi will offer vertical capability **Packs** — domain bundles of cloud service families plus an optional real backing engine:

- **Core Pack** — the base cloud APIs (what ADR-001 calls "the conformance pack").
- **AI Pack** — Bedrock / Vertex AI / Azure AI, backed privately by **Ollama** (and WebLLM in the browser).
- **IoT Pack, Security / Identity / Compliance Pack, ML Pack, ...**

This adds a second product axis — **Pack × Tier** — which could explode into dozens of hand-built SKUs and tempt a fork per pack.

Decision. Packs are **composable modules**, never pre-built bundles or forks. A pack = {service-family conformance handlers} + {an optional real backing engine}. Users pick a **tier** and **enable packs**; the build/runtime assembles the combination. **Compose, don't pre-build.**

Mechanism.

- A pack's **real engine** (Ollama for AI, a device sim for IoT, a policy engine for Security) is just another **BackendProvider**. The + /Max-vs-Lite split repeats inside every pack with the same meaning: engine on (+ /Max) vs API/SDK conformance only (Lite).
- A pack's **handlers** (e.g. the Bedrock/Vertex/Azure-AI SDK surface) are **service-family modules** registered onto the core, not hard-wired.
- **Pack selection is a build/runtime flag** — the natural extension of ADR-001's "compute = a runtime flag." One artifact per tier; packs toggle in. No per-combination artifact.

WASM parallel. Even AI has a zero-install tier: **AI-Pack-Nano+** runs in-browser inference via **WebLLM / transformers.js** (the AI analog of CheerpJ/Pyodide for compute); **AI-Pack-Nano** validates SDK shape only.

Invariants (extend ADR-001).

1. **Conformance per pack, per tier.** Each pack proves its native cloud SDKs (Bedrock/Vertex/Azure-AI, ...) as-is; the backing engine (Ollama/WebLLM) is our private pick, never surfaced. A pack/tier combo with no engine implementation is marked *unsupported*, never reported green.
2. **Packs are additive, not invasive.** A pack registers handlers + an optional **BackendProvider**; it must not modify the core or other packs. CI gate: removing a pack leaves the rest building and passing.
3. **WASM-clean still applies** to any pack targeting Nano — its handlers must be Pyodide-portable; its engine (if any) is an in-browser WASM runtime.

System-requirements impact. Requirements become **tier baseline + pack overhead**. Real engines are heavy — Ollama + a 7B model $\approx$ **+5–8 GB RAM and several GB disk** — so AI-Pack-Max realistically wants 24–32 GB while AI-Pack-Nano needs almost nothing. The requirements table gains a per-pack adder.

Rollout (roadmap). Core Pack ships first (it is the current product). Then AI Pack (Ollama / WebLLM), then IoT / Security-Identity-Compliance / ML — order driven by demand and engine maturity.

Consequences.

- (+) New domains = new modules, not new repos or SKUs; the matrix scales by composition.
- (+) Conformance integrity extends cleanly to every pack.
- (–) Requires a real pack-registration/module system + a per-pack conformance suite.
- (–) Combinatorial test surface (pack $\times$ tier $\times$ cloud) — mitigated by composition + per-pack CI, not exhaustive bundle builds.

Rejected alternatives.

- *Pre-built bundle per Pack $\times$ Tier combination* — dozens of artifacts to build and maintain; drift. No.
 - *A fork/repo per pack* — same conformance-integrity and maintenance failure as ADR-001's rejected forking. No.
-

The sections below describe the **CloudMax** tier specifically (its Multipass launcher, packaging, and runtime layout). CloudLite reuses the same core via the repo's host `docker-compose.yml` ; CloudNano runs that core in Pyodide (see `docs/browser-lite/`).

Recommended Shape

The cleanest operational model is:

1. The host installs a small launcher and Multipass.
2. The launcher creates one durable Multipass VM appliance.
3. The VM runs the full Vyomi runtime.
4. The host only starts, stops, and updates the appliance.

This gives us:

- one installation story,
- one restart boundary,
- and one place where the durable simulation state lives.

Single Launcher

Vyomi now has one launcher path:

```
bash ./scripts/cloud-learn up
```

or on Windows:

```
.\scripts\cloud-learn.ps1 up
```

The launcher starts one durable Multipass VM appliance and then starts the full Vyomi runtime inside that VM from `docker-compose.appliance.yml` .

Packaging

The packaging targets are wrappers around the same launcher:

- Homebrew
- Snap
- MSI / winget

They all bring up the same appliance boundary and should not reintroduce a separate developer runtime path.

What runs inside the VM:

- simulator UI/API

- CloudSim backbone
- provider emulators
- VM-local runtime bridge
- persistent state
- EC2-like sandboxes backed by LXD

Why:

- the VM becomes the durable boundary,
- host restarts are much less disruptive,
- and the platform no longer depends on host-native runtime wiring after startup.

What Should Live Where

Host

Keep the host thin:

- launcher
- VM lifecycle management
- optional port forwarding or SSH tunnel
- package-manager integration

Do not keep the business logic on the host.

Inside the VM

Keep the runtime self-contained:

- simulator
- CloudSim
- providers
- sandbox runtime
- deployment workspaces
- persistent state

What To Use For Runtime Orchestration

Recommended:

- `docker compose` inside the VM for the simulator stack
- or `systemd` if you want tighter OS integration

Avoid:

- Multipass inside Multipass
- laptop-side runtime bridges as a long-term dependency

User Experience

The target UX should be:

1. Install once.
2. Start one command or one app.
3. Open a browser.
4. Create resources locally.
5. Export Terraform.
6. Deploy to real AWS/GCP/Azure when ready.

Implementation Order

1. Make the appliance VM the first-class runtime.
2. Move durable state into the VM.
3. Start the full runtime stack inside the VM.
4. Keep the host launcher thin.
5. Add installer packaging per OS.
6. Add Terraform export/import workflows.

Rollout Checklist

This is the execution checklist for the distribution workstream.

- ☒ Define the distribution modes: developer, appliance, end-user.
- ☒ Persist host OS and bridge metadata in the launcher-written host config.
- ☒ Make the launcher expose a distribution mode field in the runtime contract.
- ☒ Add a first-class appliance bootstrap path that starts the full stack inside one VM.
- ☒ Wire the launcher to route top-level lifecycle commands through appliance mode.
- ☒ Make appliance runtime detection ignore host bridge/config and use appliance-local state.
- ☒ Add package-manager metadata for macOS, Linux, and Windows installers.
- ☒ Add a VM bootstrap health check that fails fast if the appliance cannot reach its internal services.
- ☒ Move the remaining runtime assumptions away from the host OS and into the appliance boundary.
- ☒ Add a clean UI entry point for the Terraform export / deploy workflow.

Recommendation Summary

- **Developer path:** `docker compose` or the launcher script.
- **End-user path:** OS-native installer or package manager.
- **Runtime path:** one Multipass VM appliance.
- **Inside the appliance:** simulator, CloudSim, provider services, and sandboxes.

This gives the simplest combination of:

- installability,

- restart safety,
- and future Terraform deployment support.